Intel SGX 데이터 봉인(Sealing) 메커니즘 코드 흐름 분석

개요

이 문서는 Intel SGX SDK의 데이터 봉인(Sealing) 메커니즘을 실제 코드 흐름을 따라가면서 상세히 분석합니다. sgx_seal_data() 함수부터 시작하여 봉인 정책 결정, EGETKEY 명령어를 통한 키 유도, AES-GCM 암호화, 그리고 봉인된 데이터 구조의 메타데이터 구성까지 전체 과정을 코드 레벨에서 추적합니다.

1. sgx_seal_data() 함수의 봉인 정책 결정 로직

1.1 메인 진입점 - sgx_seal_data()

파일 위치: `sdk/tseal/tSeal.cpp`

```
extern "C" sgx_status_t sgx_seal_data(const uint32_t additional_MACtext_leng
const uint8_t *p_additional_MACtext,
const uint32_t text2encrypt_length,
const uint8_t *p_text2encrypt,
const uint32_t sealed_data_size,
sgx_sealed_data_t *p_sealed_data)
{
sgx_status_t err = SGX_ERROR_UNEXPECTED;
sgx_attributes_t attribute_mask;
```

```
attribute_mask.flags = TSEAL_DEFAULT_FLAGSMASK;

attribute_mask.xfrm = 0x0;

uint16_t key_policy = SGX_KEYPOLICY_MRSIGNER;



// KSS(Key Separation and Sharing) 지원 확인

const  sgx_report_t* report = sgx_self_report();

if (report->body.attributes.flags & SGX_FLAGS_KSS)

{

key_policy = SGX_KEYPOLICY_MRSIGNER | KEY_POLICY_KSS;

}



// 확장된 봉인 함수 호출

err = sgx_seal_data_ex(key_policy, attribute_mask, TSEAL_DEFAULT_MISCMASK,

additional_MACtext_length, p_additional_MACtext,

text2encrypt_length, p_text2encrypt,

sealed_data_size, p_sealed_data);

return err;

}
```

핵심 분석 포인트:

- **기본 봉인 정책**: `SGX_KEYPOLICY_MRSIGNER` (서명자 기반 봉인)
- **KSS 지원 검사**: `sgx_self_report()` 를 통해 현재 Enclave의 속성 확인
- **동적 정책 설정**: KSS 지원 시 `KEY_POLICY_KSS` 플래그 추가

1.2 확장 봉인 함수 - sgx_seal_data_ex()

```
extern "C" sgx_status_t sgx_seal_data_ex(const uint16_t key_policy,
const sgx_attributes_t attribute_mask,
const sgx_misc_select_t misc_mask,
const uint32_t additional_MACtext_length,
const uint8_t *p_additional_MACtext,
const uint32_t text2encrypt_length,
const uint8_t *p_text2encrypt,
const uint32_t sealed_data_size,
sgx_sealed_data_t *p_sealed_data)
{
// 1. 매개변수 검증
uint32_t sealedDataSize = sgx_calc_sealed_data_size(additional_MACtext_length, t
if (sealedDataSize == UINT32_MAX) {
return SGX_ERROR_INVALID_PARAMETER;
}

// 2. 키 정책 검증
if ((key_policy & ~(SGX_KEYPOLICY_MRENCLAVE | SGX_KEYPOLICY_MRSIGNER |
(KEY_POLICY_KSS) | SGX_KEYPOLICY_NOISVPRODID)) ||
(key_policy & (SGX_KEYPOLICY_MRENCLAVE | SGX_KEYPOLICY_MRSIGNER)) == 0)
{
```

```
return SGX_ERROR_INVALID_PARAMETER;

}



// 3. 키 요청 구조체 구성

sgx_key_request_t tmp_key_request;

memset(&tmp_key_request, 0, sizeof(sgx_key_request_t));

const  sgx_report_t *report = sgx_self_report();

// CPU SVN과 ISV SVN 복사

memcpy(&(tmp_key_request.cpu_svn), &(report->body.cpu_svn), sizeof(sgx_cpu_svn_t

memcpy(&(tmp_key_request.isv_svn), &(report->body.isv_svn), sizeof(sgx_isv_svn_t

tmp_key_request.config_svn = report->body.config_svn;

tmp_key_request.key_name = SGX_KEYSELECT_SEAL; // 봉인 키 선택

tmp_key_request.key_policy = key_policy;

tmp_key_request.attribute_mask.flags = attribute_mask.flags;

tmp_key_request.attribute_mask.xfrm = attribute_mask.xfrm;

tmp_key_request.misc_mask = misc_mask;



// 4. 랜덤 키 ID 생성

sgx_key_id_t keyID;

err = sgx_read_rand(reinterpret_cast<uint8_t *>(&keyID), sizeof(sgx_key_id_t));

if (err != SGX_SUCCESS) {

goto clear_return;

}
```

```
memcpy(&(tmp_key_request.key_id), &keyID, sizeof(sgx_key_id_t));


// 5. 실제 봉인 수행
uint8_t  payload_iv[SGX_SEAL_IV_SIZE];
memset(&payload_iv, 0, sizeof(payload_iv));
err = random_stack_advance<0x400>(sgx_seal_data_iv, additional_MACtext_length,
p_additional_MACtext, text2encrypt_length,
p_text2encrypt, payload_iv,
&tmp_key_request, p_sealed_data);


if (err == SGX_SUCCESS) {
// 키 요청 정보를 봉인된 데이터에 복사
memcpy(&(p_sealed_data->key_request), &tmp_key_request, sizeof(sgx_key_request_t
}


clear_return:
// 임시 상태 정리
memset_s(&keyID, sizeof(sgx_key_id_t), 0, sizeof(sgx_key_id_t));
return err;
}
```

핵심 분석 포인트:

- **키 정책 검증**: MRENCLAVE 또는 MRSIGNER 중 하나는 반드시 설정

- **Report 기반 정보 수집**: 현재 Enclave의 측정값과 보안 버전 정보 획득
- **키 요청 구조체 구성**: EGETKEY 명령어에 사용될 매개변수 준비
- **스택 랜덤화**: `random_stack_advance<0x400>()` 사용으로 사이드 채널 공격 방지

2. EGETKEY 명령어 처리와 키 유도 과정

2.1 sgx_get_key() 래퍼 함수

파일 위치: `sdk/selib/sgx_get_key.cpp`

```
sgx_status_t  sgx_get_key(const  sgx_key_request_t  *key_request, sgx_key_128bit

{

sgx_status_t err = SGX_ERROR_UNEXPECTED;

void *buffer = NULL;

size_t size = 0, buf_ptr = 0;

sgx_key_request_t *tmp_key_request = NULL;

sgx_key_128bit_t *tmp_key = NULL;

egetkey_status_t egetkey_status = EGETKEY_SUCCESS;



// 1. 매개변수 검증

if(!key_request || !sgx_is_within_enclave(key_request, sizeof(*key_request))) {

err = SGX_ERROR_INVALID_PARAMETER;

goto CLEANUP;

}
```

```
if(!key || !sgx_is_within_enclave(key, sizeof(*key))) {
err = SGX_ERROR_INVALID_PARAMETER;
goto CLEANUP;
}

// 2. 정렬된 메모리 할당
// key_request: 512바이트 정렬, key: 16바이트 정렬
size = ROUND_TO(sizeof(*key_request), KEY_REQUEST_ALIGN_SIZE) +
ROUND_TO(sizeof(*key), KEY_ALIGN_SIZE);
size += MAX(KEY_REQUEST_ALIGN_SIZE, KEY_ALIGN_SIZE) - 1;

buffer = malloc(size);
if(buffer == NULL) {
err = SGX_ERROR_OUT_OF_MEMORY;
goto CLEANUP;
}
memset(buffer, 0, size);

// 3. 정렬된 포인터 설정
buf_ptr = reinterpret_cast<size_t>(buffer);
buf_ptr = ROUND_TO(buf_ptr, KEY_REQUEST_ALIGN_SIZE);
tmp_key_request = reinterpret_cast<sgx_key_request_t *>(buf_ptr);
buf_ptr += sizeof(*tmp_key_request);
```

```
buf_ptr = ROUND_TO(buf_ptr, KEY_ALIGN_SIZE);

tmp_key = reinterpret_cast<sgx_key_128bit_t *>(buf_ptr);



// 4. 데이터 복사 및 EGETKEY 호출

memcpy_s(tmp_key_request, sizeof(*tmp_key_request), key_request, sizeof(*key_req

egetkey_status = (egetkey_status_t) do_egetkey(tmp_key_request, tmp_key);

// 5. 에러 코드 변환

switch(egetkey_status) {

case EGETKEY_SUCCESS:

err = SGX_SUCCESS;

break;

case EGETKEY_INVALID_ATTRIBUTE:

err = SGX_ERROR_INVALID_ATTRIBUTE;

break;

case EGETKEY_INVALID_CPUSVN:

err = SGX_ERROR_INVALID_CPUSVN;

break;

case EGETKEY_INVALID_ISVSVN:

err = SGX_ERROR_INVALID_ISVSVN;

break;

case EGETKEY_INVALID_KEYNAME:

err = SGX_ERROR_INVALID_KEYNAME;

break;
```

```
default:
err = SGX_ERROR_UNEXPECTED;
break;
}


CLEANUP:
if((SGX_SUCCESS != err) && (NULL != key)) {
// 실패 시 키 버퍼를 랜덤 데이터로 채움
sgx_read_rand(reinterpret_cast<uint8_t *>(key), sizeof(*key));
}
else if(NULL != key) {
// 성공 시 키 복사
memcpy_s(key, sizeof(*key), tmp_key, sizeof(*tmp_key));
}


// 메모리 정리
if(buffer) {
memset_s(buffer, size, 0, size);
free(buffer);
}


return err;
```

```
}
```

2.2 do_egetkey() 어셈블리 래퍼

파일 위치: `sdk/trts/linux/trts_pic.S`

```
/*
* extern "C" int do_egetkey(key_request_t *key_request, key_128bit_t *key)
* return value:
* 0 - success
* none-zero - EGETKEY error code
* EGETKEY: rbx - the address of KEYREQUEST structure
* rcx - the address where the key is outputted
*/
DECLARE_LOCAL_FUNC do_egetkey
SE_PROLOG
mov $SE_EGETKEY, %xax /* EGETKEY leaf */
ENCLU /* EGETKEY 명령어 실행 */
#ifdef SE_SIM
cmp $SGX_SUCCESS, %xax /* 시뮬레이션 모드에서는 ZF 플래그 확인 불가 */
jnz .Legetkey_done /* 스택 정리 작업이 ZF 플래그를 항상 클리어하기 때문 */
#else
jz .Legetkey_done /* EGETKEY 에러 시 ZF 플래그 설정, 에러 코드는 xax에 */
#endif
```

```
xor %xax, %xax /* 성공 시 0 반환 */

.Legetkey_done:

SE_EPILOG

END_FUNC
```

핵심 분석 포인트:

- **레지스터 사용**: RBX에 key_request 주소, RCX에 key 출력 주소
- **ENCLU 명령어**: SE_EGETKEY 리프로 하드웨어 키 유도 실행
- **에러 처리**: 하드웨어는 ZF 플래그로 성공/실패 표시, 에러 코드는 RAX에 저장
- **시뮬레이션 모드**: ZF 플래그 신뢰성 문제로 반환 코드 직접 확인

2.3 키 유도 과정의 하드웨어 구현 (시뮬레이션)

파일 위치: `sdk/simulation/tinst/t_instructions.cpp`

```
static int _EGETKEY(sgx_key_request_t* kr, sgx_key_128bit_t okey)

{

// 1. 키 요청 구조체 정렬 검사

if (!is_aligned(kr, KEY_REQUEST_ALIGN_SIZE)) {

return EGETKEY_INVALID_KEYNAME;

}



// 2. 현재 Enclave의 SECS 정보 획득

CEnclaveMngr *mngr = CEnclaveMngr::get_instance();

CEnclaveSim *ce = mngr->get_enclave(get_enclave_id());
```

```
if (ce == NULL) {
return EGETKEY_INVALID_KEYNAME;
}
secs_t *secs = &ce->get_secs();


// 3. CPU SVN 검증
if (memcmp(&kr->cpu_svn, &DEFAULT_CPUSVN, sizeof(DEFAULT_CPUSVN)) &&
memcmp(&kr->cpu_svn, &DOWNGRADED_CPUSVN, sizeof(DOWNGRADED_CPUSVN))) {
return EGETKEY_INVALID_CPUSVN;
}


// 4. ISV SVN 검증
if (kr->isv_svn > secs->isv_svn) {
return EGETKEY_INVALID_ISVSVN;
}


// 5. 키 이름별 처리
switch (kr->key_name) {
case SGX_KEYSELECT_SEAL:
// 봉인 키 유도
if (kr->key_policy & SGX_KEYPOLICY_MRENCLAVE) {
// MRENCLAVE 기반 키 유도
derive_key(secs->mr_enclave, kr, okey);
```

```
} else if (kr->key_policy & SGX_KEYPOLICY_MRSIGNER) {

// MRSIGNER 기반 키 유도

derive_key(secs->mr_signer, kr, okey);

}

break;

case SGX_KEYSELECT_REPORT:

// Report 키 유도

derive_report_key(secs, kr, okey);

break;

default:

return EGETKEY_INVALID_KEYNAME;

}



return EGETKEY_SUCCESS;

}
```

핵심 분석 포인트:

- **정렬 요구사항**: 키 요청 구조체는 512바이트 정렬 필수
- **보안 버전 검증**: CPU SVN과 ISV SVN이 현재 Enclave보다 높으면 거부
- **키 정책 적용**: MRENCLAVE vs MRSIGNER 기반 키 유도 분기
- **키 유도 알고리즘**: 측정값과 키 요청 정보를 조합한 암호학적 해시

3. AES-GCM 암호화 과정

3.1 sgx_seal_data_iv() 내부 암호화 로직

파일 위치: `sdk/tseal/tSeal_internal.cpp`

```
sgx_status_t sgx_seal_data_iv(const uint32_t additional_MACtext_length,
const uint8_t *p_additional_MACtext, const uint32_t text2encrypt_length,
const uint8_t *p_text2encrypt, const uint8_t *p_payload_iv,
const sgx_key_request_t* p_key_request, sgx_sealed_data_t *p_sealed_data)
{
sgx_status_t err = SGX_ERROR_UNEXPECTED;


// 1. 봉인 키 생성 (스택 랜덤화 적용)
using cseal_key200 = randomly_placed_object<
sgx::custom_alignment_aligned<sgx_key_128bit_t, sizeof(sgx_key_128bit_t), 0, siz
0x200>;
cseal_key200 oseal_key_buf;
auto* oseal_key = oseal_key_buf.instantiate_object();
auto* seal_key = &oseal_key->v;
err = sgx_get_key(p_key_request, seal_key);
if (err != SGX_SUCCESS) {
if (err != SGX_ERROR_OUT_OF_MEMORY)
err = SGX_ERROR_UNEXPECTED;
return err;
}
```

```
// 2. AES-GCM 암호화 수행 (스택 랜덤화 적용)
err = random_stack_advance<0x200>(sgx_rijndael128GCM_encrypt,
seal_key, // 128비트 봉인 키
p_text2encrypt, // 암호화할 데이터
text2encrypt_length, // 데이터 길이
reinterpret_cast<uint8_t *>(&(p_sealed_data->aes_data.payload)), // 출력 버퍼
p_payload_iv, // 초기화 벡터 (12바이트)
SGX_SEAL_IV_SIZE, // IV 길이
p_additional_MACtext, // 추가 인증 데이터 (AAD)
additional_MACtext_length, // AAD 길이
&(p_sealed_data->aes_data.payload_tag)); // 출력 MAC 태그

if (err == SGX_SUCCESS) {
// 3. 추가 MAC 텍스트 복사
uint8_t* p_aad = NULL;
if (additional_MACtext_length > 0) {
p_aad = &(p_sealed_data->aes_data.payload[text2encrypt_length]);
memcpy(p_aad, p_additional_MACtext, additional_MACtext_length);
}

// 4. 메타데이터 설정
p_sealed_data->plain_text_offset = text2encrypt_length;
```

```
p_sealed_data->aes_data.payload_size = additional_MACtext_length + text2encrypt_

}



return err;

}
```

3.2 sgx_rijndael128GCM_encrypt() 구현

파일 위치: `sdk/tlibcrypto/sgxssl/sgx_aes_gcm.cpp`

```
sgx_status_t sgx_rijndael128GCM_encrypt(const sgx_aes_gcm_128bit_key_t *p_key,

const uint8_t *p_src, uint32_t src_len,

uint8_t *p_dst, const uint8_t *p_iv, uint32_t iv_len,

const uint8_t *p_aad, uint32_t aad_len,

sgx_aes_gcm_128bit_tag_t *p_out_mac)

{

return aes_gcm_encrypt_internal((const uint8_t *)p_key, sizeof(sgx_aes_ctr_128bi

p_src, src_len, p_dst, p_iv, iv_len,

p_aad, aad_len, p_out_mac);

}
```

3.3 aes_gcm_encrypt_internal() 상세 구현

```
static sgx_status_t aes_gcm_encrypt_internal(const uint8_t *p_key, uint32_t key_
```

```
const uint8_t *p_src, uint32_t src_len,
uint8_t *p_dst, const uint8_t *p_iv, uint32_t iv_len,
const uint8_t *p_aad, uint32_t aad_len,
sgx_aes_gcm_128bit_tag_t *p_out_mac)
{
// 1. 매개변수 검증
if ((src_len >= INT_MAX) || (aad_len >= INT_MAX) || (key_len != SGX_AESGCM_KEY_S
((src_len > 0) && (p_dst == NULL)) || (p_key == NULL) || (p_iv == NULL) ||
(iv_len != SGX_AESGCM_IV_SIZE) || ((aad_len > 0) && (p_aad == NULL)) ||
(p_out_mac == NULL)) {
return SGX_ERROR_INVALID_PARAMETER;
}


sgx_status_t ret = SGX_ERROR_UNEXPECTED;
EVP_CIPHER_CTX *pState = NULL;
int len = 0;


do {
// 2. OpenSSL 컨텍스트 생성
if (!(pState = EVP_CIPHER_CTX_new())) {
ret = SGX_ERROR_OUT_OF_MEMORY;
break;
}
```

```
// 3. AES-128-GCM 초기화
if (!EVP_EncryptInit_ex(pState, EVP_aes_128_gcm(), NULL, NULL, NULL)) {
break;
}

// 4. IV 길이 설정
if (!EVP_CIPHER_CTX_ctrl(pState, EVP_CTRL_GCM_SET_IVLEN, iv_len, NULL)) {
break;
}

// 5. 키와 IV 설정
if (!EVP_EncryptInit_ex(pState, NULL, NULL, p_key, p_iv)) {
break;
}

// 6. 추가 인증 데이터(AAD) 처리
if (aad_len > 0) {
if (!EVP_EncryptUpdate(pState, NULL, &len, p_aad, aad_len)) {
break;
}
}
```

```
// 7. 실제 데이터 암호화
if (src_len > 0) {
if (!EVP_EncryptUpdate(pState, p_dst, &len, p_src, src_len)) {
break;
}
}

// 8. 암호화 완료
if (!EVP_EncryptFinal_ex(pState, p_dst + len, &len)) {
break;
}

// 9. GCM 태그 추출
if (!EVP_CIPHER_CTX_ctrl(pState, EVP_CTRL_GCM_GET_TAG, SGX_AESGCM_MAC_SIZE, p_ou
break;
}

ret = SGX_SUCCESS;
} while (0);

// 10. 리소스 정리
if (pState) {
EVP_CIPHER_CTX_free(pState);
```

```
}

return ret;
}
```

핵심 분석 포인트:

- **OpenSSL 기반**: EVP 인터페이스를 통한 AES-GCM 구현
- **GCM 모드**: 암호화와 인증을 동시에 제공하는 AEAD 모드
- **AAD 처리**: 암호화되지 않지만 인증에 포함되는 추가 데이터
- **태그 생성**: 128비트 인증 태그로 무결성 보장

4. 봉인된 데이터 구조와 메타데이터

4.1 sgx_sealed_data_t 구조체 정의

파일 위치: `common/inc/sgx_tseal.h`

```
typedef struct _aes_gcm_data_t
{
uint32_t payload_size; /* 0: 암호화된 데이터와 추가 MAC 텍스트를 포함한 페이로드 크기 *
uint8_t reserved[12]; /* 4: 예약된 비트 */
uint8_t payload_tag[SGX_SEAL_TAG_SIZE]; /* 16: 평문, 페이로드, 크기의 AES-GMAC */
uint8_t payload[]; /* 32: 암호화된 데이터와 추가 MAC 텍스트를 포함한 페이로드 데이터 */
} sgx_aes_gcm_data_t;
```

```
typedef struct _sealed_data_t

{

sgx_key_request_t key_request; /* 00: 봉인 키 획득에 사용된 키 요청 */

uint32_t plain_text_offset; /* 64: aes_data.payload 내에서 추가 MAC 텍스트 시작 오프

uint8_t reserved[12]; /* 68: 예약된 비트 */

sgx_aes_gcm_data_t aes_data; /* 80: AES/GCM 관련 데이터를 담는 구조체 */

} sgx_sealed_data_t;
```

4.2 메타데이터 구성 과정

```
// sgx_seal_data_iv() 함수 내에서 메타데이터 설정

if (err == SGX_SUCCESS) {

// 1. 추가 MAC 텍스트 복사

uint8_t* p_aad = NULL;

if (additional_MACtext_length > 0) {

p_aad = &(p_sealed_data->aes_data.payload[text2encrypt_length]);

memcpy(p_aad, p_additional_MACtext, additional_MACtext_length);

}


// 2. 메타데이터 필드 설정

p_sealed_data->plain_text_offset = text2encrypt_length; // 암호화된 데이터 길이

p_sealed_data->aes_data.payload_size = additional_MACtext_length + text2encrypt_

}
```

```
// sgx_seal_data_ex() 함수에서 키 요청 정보 복사
if (err == SGX_SUCCESS) {
memcpy(&(p_sealed_data->key_request), &tmp_key_request, sizeof(sgx_key_request_t
}
```

4.3 봉인된 데이터 레이아웃

```
sgx_sealed_data_t 구조:
┌──────────────────────────────────────────────────────────────────┐
| sgx_key_request_t key_request (64 bytes) |
| ┌──────────────────────────────────────────────────────────┐ |
| | uint16_t key_name = SGX_KEYSELECT_SEAL | |
| | uint16_t key_policy (MRENCLAVE/MRSIGNER) | |
| | sgx_isv_svn_t isv_svn | |
| | sgx_cpu_svn_t cpu_svn | |
| | sgx_attributes_t attribute_mask | |
| | sgx_key_id_t key_id (random) | |
| | sgx_misc_select_t misc_mask | |
| └──────────────────────────────────────────────────────────┘ |
├──────────────────────────────────────────────────────────────────┤
| uint32_t plain_text_offset (4 bytes) |
```

```
├──────────────────────────────────────────────────────────────────────────┤
| uint8_t reserved[12] (12 bytes) |
├──────────────────────────────────────────────────────────────────────────┤
| sgx_aes_gcm_data_t aes_data |
| ┌──────────────────────────────────────────────────────────────────────┐ |
| | uint32_t payload_size (4 bytes) | |
| | uint8_t reserved[12] (12 bytes) | |
| | uint8_t payload_tag[16] (AES-GCM MAC tag) | |
| | ┌──────────────────────────────────────────────────────────────────┐ | |
| | | uint8_t payload[] | | |
| | | ┌──────────────────────────────────────────────────────────────┐ | | |
| | | | Encrypted Data (text2encrypt_length bytes) | | | |
| | | └──────────────────────────────────────────────────────────────┘ | | |
| | | ┌──────────────────────────────────────────────────────────────┐ | | |
| | | | Additional MAC Text (additional_MACtext_length) | | | |
| | | └──────────────────────────────────────────────────────────────┘ | | |
| | └──────────────────────────────────────────────────────────────────┘ | |
| └──────────────────────────────────────────────────────────────────────┘ |
└──────────────────────────────────────────────────────────────────────────┘
```

4.4 무결성 검증 메커니즘

4.4.1 봉인 시 무결성 보장

```
// AES-GCM 암호화 과정에서 자동으로 MAC 태그 생성
err = sgx_rijndael128GCM_encrypt(seal_key, // 봉인 키
p_text2encrypt, // 암호화할 데이터
text2encrypt_length, // 데이터 길이
encrypted_output, // 암호화된 출력
p_payload_iv, // IV
SGX_SEAL_IV_SIZE, // IV 길이
p_additional_MACtext, // AAD
additional_MACtext_length, // AAD 길이
&payload_tag); // 출력 MAC 태그
```

4.4.2 해제 시 무결성 검증

파일 위치: `sdk/tseal/tSeal_internal.cpp`

```
sgx_status_t sgx_unseal_data_helper(const sgx_sealed_data_t *p_sealed_data,
uint8_t *p_additional_MACtext,
uint32_t additional_MACtext_length,
uint8_t *p_decrypted_text,
uint32_t decrypted_text_length)
{
// 1. 봉인 키 재생성
using cseal_key200 = randomly_placed_object<
sgx::custom_alignment_aligned<sgx_key_128bit_t, sizeof(sgx_key_128bit_t), 0, siz
```

```
0x200>;
cseal_key200 oseal_key_buf;
auto* oseal_key = oseal_key_buf.instantiate_object();
auto* seal_key = &oseal_key->v;
err = sgx_get_key(&p_sealed_data->key_request, seal_key);
if (err != SGX_SUCCESS) {
if ((err == SGX_ERROR_INVALID_CPUSVN) || (err == SGX_ERROR_INVALID_ISVSVN) ||
(err == SGX_ERROR_OUT_OF_MEMORY))
return err;
return SGX_ERROR_MAC_MISMATCH; // 키 유도 실패 시 무결성 오류로 처리
}

// 2. 사이드 채널 공격 방지를 위한 메모리 펜스
sgx_lfence();

// 3. AES-GCM 복호화 및 무결성 검증
uint8_t payload_iv[SGX_SEAL_IV_SIZE];
memset(&payload_iv, 0, SGX_SEAL_IV_SIZE);
err = random_stack_advance<0x200>(sgx_rijndael128GCM_decrypt,
seal_key, // 봉인 키
const_cast<uint8_t *>(p_sealed_data->aes_data.payload), // 암호화된 데이터
decrypted_text_length, // 복호화할 길이
p_decrypted_text, // 복호화 출력
```

```
&payload_iv[0], // IV
SGX_SEAL_IV_SIZE, // IV 길이
const_cast<uint8_t *>(&(p_sealed_data->aes_data.payload[decrypted_text_length]))
additional_MACtext_length, // AAD 길이
const_cast<sgx_aes_gcm_128bit_tag_t *>(&p_sealed_data->aes_data.payload_tag)); /

if (err != SGX_SUCCESS) {
return err; // MAC 검증 실패 시 SGX_ERROR_MAC_MISMATCH 반환
}

// 4. 추가 MAC 텍스트 복사
if (additional_MACtext_length > 0) {
memcpy(p_additional_MACtext, &(p_sealed_data->aes_data.payload[decrypted_text_le
additional_MACtext_length);
}

return SGX_SUCCESS;
}
```

5. 보안 고려사항 및 사이드 채널 공격 방지

5.1 스택 랜덤화 (Stack Randomization)

```
// random_stack_advance 템플릿 사용으로 스택 레이아웃 랜덤화
template<size_t N, typename F, typename... Args>
auto random_stack_advance(F&& f, Args&&... args) -> decltype(f(args...)) {
// N 바이트만큼 스택을 랜덤하게 이동시켜 사이드 채널 공격 방지
uint8_t dummy[N];
sgx_read_rand(dummy, sizeof(dummy));
return f(std::forward<Args>(args)...);
}

// 사용 예시
err = random_stack_advance<0x400>(sgx_seal_data_iv, ...); // 1KB 스택 랜덤화
err = random_stack_advance<0x200>(sgx_rijndael128GCM_encrypt, ...); // 512B 스택
```

5.2 메모리 정렬 요구사항

```
// 키 요청 구조체는 512바이트 정렬 필수
#define KEY_REQUEST_ALIGN_SIZE 512
#define KEY_ALIGN_SIZE 16

// 정렬된 메모리 할당
buf_ptr = ROUND_TO(buf_ptr, KEY_REQUEST_ALIGN_SIZE);
tmp_key_request = reinterpret_cast<sgx_key_request_t *>(buf_ptr);
```

5.3 메모리 정리 및 보안

```
// 민감한 데이터 안전한 정리
memset_s(&keyID, sizeof(sgx_key_id_t), 0, sizeof(sgx_key_id_t));
memset_s(buffer, size, 0, size);

// 실패 시 키 버퍼를 랜덤 데이터로 채움
if((SGX_SUCCESS != err) && (NULL != key)) {
sgx_read_rand(reinterpret_cast<uint8_t *>(key), sizeof(*key));
}
```

결론

Intel SGX의 데이터 봉인 메커니즘은 다음과 같은 단계로 구성됩니다:

1. **봉인 정책 결정**: MRSIGNER 기반 기본 정책, KSS 지원 시 확장
2. **키 유도**: EGETKEY 명령어를 통한 하드웨어 기반 키 생성
3. **AES-GCM 암호화**: OpenSSL 기반 AEAD 암호화로 기밀성과 무결성 동시 보장
4. **메타데이터 구성**: 키 요청 정보와 페이로드 구조 정보 저장
5. **보안 강화**: 스택 랜덤화, 메모리 정렬, 안전한 메모리 정리

이 전체 과정은 하드웨어 보안 기능과 소프트웨어 암호화 라이브러리를 조합하여 강력한 데이터 보호 메커니즘을 제공합니다.